Building with AI

The take-home guide for building real apps with AI builders like Replit, Lovable, or Base44.

Most people jump straight to *"build me an app."* The AI then guesses everything (who it's for, what it looks like, what it shouldn't do) and the guesses are usually wrong. **This guide fixes that.** Part 1 covers the basics that apply to any AI builder. Part 2 covers Replit specifics, from budget to launch.

PART 1 — THE BASICS

1. Anatomy of an app

Every app you'll build has the same four pieces. You don't have to code them — but you do need to know which one you're pointing the AI at.

- **Frontend** — everything the user sees and taps: screens, buttons, forms. *"The button is ugly"* is a frontend problem.
- **Backend** — the logic that runs on the server: who's allowed to do what, what happens when someone books or pays. *"Stripe isn't charging"* is a backend problem.
- **Database** — where the app remembers things: users, listings, bookings. *"It forgot my data"* is a database problem.
- **Secrets** — the keys that unlock paid services (Stripe, login, the database itself). They live on the server as environment variables, never in frontend code.

> This is why your first message asks the AI to create an **ARCHITECTURE.md** file: a plain-language map of which piece is which in *your* app. When something breaks, read it first — you'll know what to ask for.

2. What's context?

Context is everything the AI needs to know *before* it can build the right thing: who the app is for, what it does, how it makes money, how it should look, and what it must *not* do. Given none of it, the AI fills every gap with a guess.

So your **very first message** in a new Replit project should not be *"build me…"*. It should be a full spec like the one below, **with photos of your storyboard and POC screens attached.**

Your storyboard and POC are the thinking you do *before* opening Replit, rough sketches of each screen and how they connect, on paper or in Figma. Photos of them carry far more intent than any paragraph of description. Have them ready before you paste.

The **yellow boxes** are what you swap with your own app's info. Everything else stays as-is. Paste the whole block as your first message, attach your storyboard and POC photos, then wait for the AI to acknowledge before you ask for any features.

FIRST MESSAGE (PASTE INTO REPLIT)

```
Build a web/native app called TuneShare, a peer-to-peer marketplace that
connects musical instrument owners with musicians who want to rent instruments.

I've attached photos of my storyboard and POC screens. Use them as the reference
for how each screen should look and how the user moves between them. The text below
covers features, monetization, branding, security, and what NOT to build.

Before you build anything, create an ARCHITECTURE.md file that explains in plain
language what the frontend is, what the backend is, what the database stores, and
how they connect. Keep it updated as the app grows.

Core features:
- Google login for sign-up
- Two user roles: Owners (list instruments) and Renters (browse and rent). A user
can be both.
- Listings: Owners post instruments with photos, description, category (guitar,
piano, drums, brass, strings), condition, hourly/weekly price, and pickup city
- Location matching: show renters the closest instruments first
- Booking & calendar: renters pick rental dates; show availability
- Handover flow: both parties confirm pickup and return
- Payments via Stripe, with a refundable deposit held until return
- In-app chat between owners and renters to coordinate handover
- Reviews and ratings for both sides after a completed rental

Monetization:
- Revenue model: 10% platform commission on every completed rental, deducted
before payout to the owner.
- Refundable security deposit (amount set per listing) held by the platform until
the return is confirmed.
- No subscription tiers in v1, pay-per-rental only. (If your app has plans, list
each tier here: name, price, what's included.)

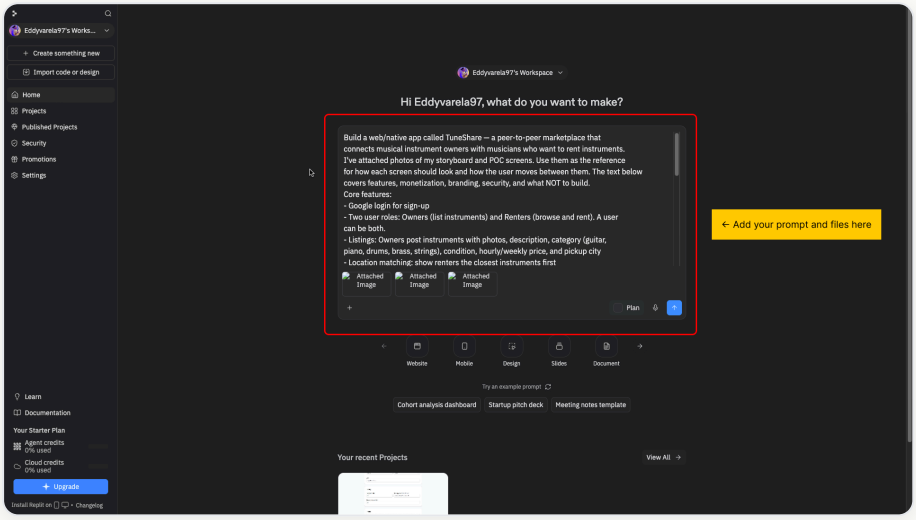
Branding & design:
- App name: TuneShare
- UI: keep it generic and functional with Tailwind UI. Use the AI builder's default
look, no custom logos, colors, or fonts. (Branding will come later, in a separate
prompt.)

Security: build this like a real app, keep secret keys (Stripe, DB, login) in
environment variables on the server, never in browser code and never hard-coded in
```

```
files; verify all user input and permissions on the server, not the client; stop
and ask me if a step feels risky.

Don't add: extra logins beyond Google, ads, long sign-up forms, paid tiers, dark
mode toggle

If anything is unclear, ask me before building.
```



This is what it looks like in Replit: paste the prompt in the message box, then attach your storyboard and POC screens.

Wait for the AI to acknowledge, then let it build.

> **Why this works:** you only say it once. Every feature you ask for afterwards inherits this context. Without it, every prompt is a fresh coin flip.

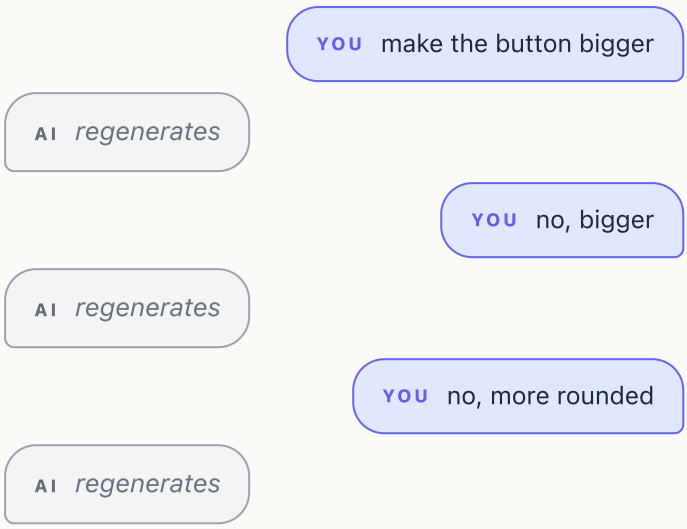
3. How to prompt?

After the first build, three habits separate people who ship from people who fight the AI for a week:

- **One change at a time, not a list.** Five requests in one message means five half-done features and no idea which one broke the app.
- **Read every reply before sending the next message.** The AI tells you what it changed and often asks a question — skipping that is how bugs slip in.
- **Be specific.** "Make it cool" gives random results. Two specific words ("warmer colors", "bigger buttons") work better than one vague one.

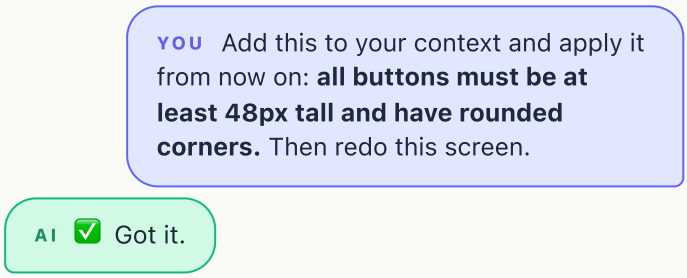
And when the output is wrong, don't argue with it — **fix the rules, not the output.** Arguing about one screen means having the same fight on the next screen, and the next.

✕ THE TRAP



...next screen the AI makes, the buttons are small and square again.

✓ THE FIX (ADD A RULE INSTEAD OF ARGUING)



Every future screen has tall, rounded buttons. For free.

A few rules people commonly end up needing:

- "Never use red, it looks like an error."
- "Always show prices with the currency symbol."
- "When something succeeds, say it in plain language, not 'Success.'"
- "Error messages must say what went wrong and what to do next — never just 'Something went wrong.'"
- "Forms have at most 3 fields per screen."

> The output is the symptom. The rules are the cause. **Fix the cause.**

4. Keep a tiny "lessons" note

Open a Google Doc called **Lessons**. Every time you find a rule you wish you'd had on day one, write one line.

Examples:

- "I have to tell the AI who the user is, every time."
- "I ask for the mobile view first; it's the harder constraint."
- "'Make it cool' gives me random results. Two specific words work better."
- "If I don't say what NOT to add, the AI adds it by default."

Next project, paste your lessons into your first message from day one. Your context gets smarter, the AI gets sharper, and you stop repeating the same fights with yourself.

PART 2 — REPLIT

5. Budget

Building a real app costs real money. Plan on the **$100/month tier** — starter credits run out fast once you're building seriously.

What burns credits: long messy conversations (the AI re-reads everything on every message), vague prompts that trigger redo loops, and asking for five features at once. The habits in this guide aren't just cleaner — they're cheaper. Check your usage regularly so the bill is never a surprise.

6. New chats

When you move on to a new feature, **start a new task** instead of continuing the old conversation.

Long conversations fill up with dead ends and old mistakes — call it *context rot*. The AI re-reads all of it on every message, so it gets more confused *and* more expensive as you go. A fresh task loses nothing important: your spec, your rules, and your ARCHITECTURE.md live in the project's files and memory, not in the chat history.

> **One feature per message. One feature per task.**

7. Artifacts

Replit doesn't just give you a chat — it gives you the whole workshop. Know your way around what it builds:

- **Preview** — your running app. This is the dev version, not the live one.
- **File tree** — the actual code. You don't need to write it, but skim **ARCHITECTURE.md** and **replit.md** (where Replit keeps its own notes about your project — your rules end up here).

- **Console / logs** — where errors show their real cause. When something breaks, look here before re-prompting; paste the error into the chat.
- **Database pane** — see the data your app is actually storing.
- **Secrets tab** — where API keys live as environment variables.
- **Deployments** — where the dev app becomes the live app, and where domains connect.

8. Security

Three habits cover most of what goes wrong:

- **Secrets stay in the Secrets tab.** API keys, database credentials, and login secrets live as environment variables on the server — never in frontend code, never hard-coded in files. (The first message already tells the AI this.)
- **Chat hygiene:** never paste real API keys, passwords, or anyone's personal data into the chat. If a key needs to go somewhere, it goes in the Secrets tab.
- **The server checks everything.** User input and permissions get verified on the backend, not the browser — anyone can edit what runs in their own browser.

> Before launch, ask the AI directly: *"Review the app for security issues: exposed keys, unvalidated input, and actions that skip permission checks. List what you find before fixing anything."*

9. Domain

Your deployed app starts at *yourapp.replit.app*. A real launch wants a real domain:

- Buy the domain (Namecheap, GoDaddy, Cloudflare — any registrar works).
- In Replit, go to your deployment's settings and link the domain — Replit gives you DNS records to add at your registrar.
- DNS changes can take hours to propagate. **Set the domain up days before launch, not launch day.**

10. Launch

The dev preview is not the live app. Before you share the link:

- ☐ **Deploy it.** Publish through Replit Deployments — the preview URL is not your product.
- ☐ **Secrets set in production.** The deployed app needs its environment variables too, not just the dev workspace.
- ☐ **Stripe is in live mode.** Swap test keys for live keys, then make one real purchase yourself, end to end, refund included.

- ☐ **Domain connected** and propagated (see above).
- ☐ **Error messages are human.** Click around and break things on purpose — every error should say what went wrong and what to do next.
- ☐ **Security pass done** — run the review prompt from the Security section.

> **Working as a team:** one person drives the AI per app — everyone else owns storyboard, content, and testing. Two people prompting the same project at once is how work gets overwritten. If you must switch drivers, finish the current task first and hand over with a one-line status.

THE WHOLE LOOP

Set the context → one feature per fresh task → if it's wrong, fix the rules not the output → write down what you learned

THAT'S IT.